

Fine-tuning a task specialist: the step-by-step playbook

Fine-tuning a task specialist: the step-by-step playbook

A repeatable recipe for building a real module the way Tracks A, B, C and the Part 2 pipeline do it: turn a knowledge source + a target task into a trained, validated adapter, without losing the model's general ability. Follow the steps in order. Each step says what to do, why, the gotcha to avoid, and where it lives in this repo as a worked example.

The shape of the whole thing:

```
decide it's the right tool -> define the task -> reserve gold ->
build seed
  -> generate synthetic data -> judge + dedup + decontaminate +
balance
  -> add a rehearsal mix -> configure the run -> train (LoRA/QLoRA)
  -> evaluate two axes -> gate (accept/reject) -> iterate -> ship
+ test
```

Step 0: make sure fine-tuning is the right tool

Fine-tuning teaches a model a **SKILL** or **BEHAVIOR**; it does not reliably store facts. Before you build anything, check you actually need it:

- The model lacks **information** (docs, current data, private data) -> use **RAG**, not fine-tuning. Facts in weights go stale and hallucinate.
- The data is **live/structured** (a database, analytics) -> query it with **SQL** (or text-to-SQL), not fine-tuning and not vector RAG.
- The model lacks a **skill/behavior**: a recurring classification/extraction task, a fixed output format, a consistent style, grounded answering/abstention, domain reasoning -> this is where fine-tuning fits.
- Best of both: **RAG for the facts + a fine-tune that uses them well** (grounded answering, RAFT). They are complements, not alternatives.

Rule of thumb: missing knowledge -> RAG/SQL. Missing skill -> fine-tune. If unsure, you probably do not need fine-tuning yet.

Step 1: define the task (contract + metric) FIRST

Write down, before touching data: - exact **input** (what the model receives), - exact **output** (format, label set, or answer shape), - exact **success metric** (accuracy / macro-F1 for classification, EM/F1 for extraction, a faithfulness judge for grounded answers).

Why: everything downstream (what to generate, what “balanced” means, how to score) is shaped by this. Without it, you cannot measure anything.

Gotcha: pick **macro-F1** when classes are imbalanced, so the dominant class can't hide weakness on rare ones.

In repo: `common.py` / `common_c.py` hold the contract (system prompt, row format, metric).

Step 2: reserve the sacred gold (evaluation) set up front

Hold out a fixed evaluation set BEFORE generating anything. Make it: - **balanced** across the axes you care about (per class / per mode), - **disjoint** from training by the right key (by document, by id, by split), - **decontaminated**: nothing in training may duplicate or near-duplicate a gold item.

Why: gold is how you prove the before/after honestly. If training leaks into gold, every number is meaningless.

Gotcha: split by the natural unit (issue id, document) not by row, or the same entity leaks into both sides. Persist the gold ids and decontaminate every later step against them.

In repo: `phase1_seed/build_seed.py` reserves gold first and asserts zero overlap.

Step 3: build a small verified seed

Pull a handful of real, verified examples from the source, in the exact task shape, balanced across classes. Small is fine (the LIMA idea: tens per class can be enough).

Why: the seed defines the task by example and is the honest control (“what does the synthetic data add over plain seed data?”). Keep the real seeds in the final mix to fight mode collapse.

In repo: `build_seed.py` writes `seed.jsonl`.

Step 4: generate synthetic data (grounded)

Expand the seed with a local model (Ollama keeps it private and cheap). The grounding rule: vary phrasing, structure, difficulty, persona; never invent the verified fact or label. - classification: keep the real label, reword the input. - grounded QA: generate questions + answers from a real passage, keep answers faithful. - if the target is a reasoning model: generate a **reasoning trace** that justifies the verified answer, and vary the trace length (short/medium/long). The label is real; only the trace is synthetic.

Why: this is the cheap way to turn a few seeds into a few hundred varied examples.

Gotcha: keep it **small** (a few hundred). Generation is one model call per item, so scale costs hours; the goal is getting the loop right, not volume. Drop any generation whose answer does not match the verified label (grounding guard).

In repo: `phase2_synthetic/gen_reasoning.py` (and Track B `qgen.py`, Track A `paraphrase/evolve`).

Step 5: quality-gate the data (judge, dedup, decontaminate, balance)

Run every candidate through gates, as pipeline steps, not by hand: 1. **LLM-as-judge** for faithfulness: does the example truly support its label, only from the given context? Keep

faithful ones (score $\geq$ a bar). 2. **dedup + near-dedup** (character-shingle Jaccard) so near-identical rows can't pile up. 3. **decontaminate** against the gold set (exact + near-dup) BEFORE training. 4. **balance** across classes (and trace-length buckets); cap the majority, keep minorities.

Why: this is what makes the dataset trustworthy at scale without hand-labeling everything.

Gotcha: the judge is often the SAME model that generated the data, so it has self-preference and length bias. Fine for a learning run; a stronger/different judge is the real fix.

In repo: `phase2_synthetic/judge.py` + `filter.py`.

Step 6: add a rehearsal (replay) mix to preserve capabilities

Mix a little general data into the task data so the model does not forget. A practical ratio is $\sim 75\%$ task / $\sim 25\%$ rehearsal, and split the rehearsal into general reasoning (e.g. GSM8K) and tool-calling examples if those abilities matter.

Why: narrow fine-tuning quietly degrades unrelated abilities (catastrophic forgetting). The replay mix is the first-class guardrail against it.

In repo: `phase2_synthetic/mix_rehearsal.py` -> `train_mix.jsonl`.

Step 7: configure the run (one config = single source of truth)

Put everything about the run in one reviewed config: base model, LoRA rank/alpha/targets, the data files (replay or not), epochs, learning rate, sequence length, the quality guardrails, and the acceptance thresholds. Rules (or later an agent) propose it; a human confirms it.

Why: config-driven runs are reproducible and automatable; the config IS the run. It also makes you set the acceptance bar up front, not after seeing results.

Gotcha: a human-confirm gate here (and at the field survey in Step 2/5) is the cheap, high-leverage place to keep quality high without hand-checking every row.

In repo: `pipeline2/config.yaml` (+ `propose_config.py`).

Step 8: train the adapter (LoRA / QLoRA)

Train a LoRA (QLoRA = 4-bit base) on the constructed set. Two disciplines that matter: - **assistant-only loss**: train on the answer/label tokens only, never the long prompt, so the signal is the target, not the input. - **no silent truncation**: set `max_len` above your longest row so the label/trace is never cut; count and warn on overflow.

Why: LoRA freezes the base (preserving general ability) and is cheap; these two settings are what make the loss land on the right thing and nothing get truncated.

Gotcha: on a small GPU, long contexts OOM. Use small batch (1) + gradient accumulation, `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True`, and run heavy training on a free Kaggle T4. Keep the model small (0.5B-3B) for fast iteration.

In repo: `phase3_train/train.py` / `train_from_config.py`; cloud cells in

Step 9: evaluate on two axes (always both)

Measure on the SAME fixed sets every time: - **axis 1 (task)**: the metric on the held-out gold set, base vs fine-tuned. - **axis 2 (regression)**: a fixed general sentinel (+ reasoning and tool probes), before and after. This catches forgetting that the task metric hides.

Why: a good fine-tune must improve the task AND not break everything else. One axis is not enough.

Gotcha: size the regression probe set so its **granularity** is meaningful. With 12 probes one flip is 0.083, so a 0.05 tolerance is unsatisfiable; use ~50-100 probes so each flip is ~1-2 points and the threshold means something.

In repo: `eval/evaluate.py` + `eval/sentinel.jsonl` / `reasoning_probes.jsonl` / `tool_probes.jsonl` .

Step 10: gate (accept or reject)

Decide accept/reject against criteria set in advance: - accept only if **task gain $\geq$ min_task_gain** AND **no probe drops more than max_regression_drop**. - on reject, apply ONE adjustment (more replay, lower LR, fewer epochs) and re-run once. - record the accepted adapter with its config and numbers, so it is reproducible.

Why: the gate is what makes the automation trustworthy. “Looks better” is not enough; it has to clear both axes by your stated bar.

Gotcha: set the bar UP FRONT and report honestly. Recalibrating after the fact is only OK for a real reason (e.g. probe granularity), not to force a pass.

In repo: `gate.py` + `pipeline.py` (the orchestrated loop).

Step 11: iterate (only if needed)

If the gate fails or the task is still weak: error-analyze the gold misses, name the pattern (a confusable pair, an under-served slice, too much forgetting), generate **targeted** data for exactly that weakness, retrain, re-measure. Stop when the metric clears the bar and the regression holds, OR when another round stops moving the number.

Why: one pass is rarely optimal, but more iterations have diminishing returns and can overfit.

Gotcha: watch for over-specialization. In Track A a second targeted round barely moved the task (+0.01) and started to regress the sentinel; that was the signal to stop, not push.

In repo: `phase5_iterate/` (error_analysis -> targeted -> build_v2 -> retrain).

Step 12: ship and test the final output

You now have an accepted adapter plus its config and eval numbers. Test it interactively before trusting it, and keep the artifacts for reproducibility.

In repo: `run_trained.py` loads base + adapter and lets you query it; `runs/report.json` records the accepted adapter, config, and verdict.

Cross-cutting disciplines (the things that actually decide quality)

- **Leakage control.** Exclude any field/feature that reveals or is determined by the outcome (final status, resolution date, outcome-coupled states). A survey + human-confirm gate is how you catch it.
 - **Keep it small and high quality.** A few hundred clean, balanced, decontaminated examples beats tens of thousands of noisy ones for LoRA specialization, and keeps iteration fast.
 - **Two human-confirm gates, not row-by-row review.** Confirm the field survey (what may the model see) and the config (the run + acceptance bar). Spend human effort at leverage points.
 - **Verified labels, synthetic phrasing.** The fact/label is always real (from the source); only the wording or the reasoning is generated.
 - **Measure regression every time.** Forgetting is invisible until you check the second axis.
-

Minimal file layout to copy for a new module

```
my_module/  
  common.py          task contract + helpers (system prompt, row format,  
metric)              seed.jsonl, gold.jsonl, train_synth.jsonl,  
  data/              train_mix.jsonl, results  
  phase1_seed/        build_seed.py          (+ survey.py if the source is  
structured)             
  phase2_synthetic/   sdg.py gen.py judge.py filter.py mix_rehearsal.py  
  phase3_train/       train.py              (LoRA/QLoRA, assistant-only  
loss)                   
  eval/              evaluate.py compare.py sentinel.jsonl +  
reasoning/tool probes   
  phase5_iterate/     error_analysis.py targeted.py build_v2.py  
(optional)             
  config.yaml         single source of truth (if you go config-driven)  
  run_trained.py      load base + adapter and test it  
  README.md PROGRESS.md
```

Compute: the practical setup

- **Generate locally** with Ollama (privacy, cheap, no scale pressure).
 - **Train on a free Kaggle/Colab T4** (a small 4 GB local GPU OOMs on long contexts). Upload the small JSONL, train, download the adapter (a few tens of MB).
 - **Small base model** (Qwen2.5-0.5B-Instruct here) so a run finishes in minutes and you can iterate the loop many times.
-

The whole thing as a checklist

- ☐ Confirmed fine-tuning is the right tool (not RAG/SQL).
- ☐ Wrote the task contract + metric.
- ☐ Reserved a balanced, decontaminated gold set (split by the right key).

- ☐ Built a small verified seed.
- ☐ Generated grounded synthetic data (varied; reasoning traces if applicable).
- ☐ Judged, deduped, decontaminated, balanced.
- ☐ Added a ~75/25 rehearsal mix.
- ☐ Wrote and confirmed the run config (acceptance bar up front).
- ☐ Trained the LoRA (assistant-only loss, max_len safe, small batch).
- ☐ Evaluated both axes (task gold + regression probes, probes sized for granularity).
- ☐ Ran the gate; did one adjusted re-run if it failed.
- ☐ Iterated only if needed; stopped at acceptance or diminishing returns.
- ☐ Tested the adapter (run_trained.py) and recorded config + numbers.